

EXPLORE / DECIDE / DESIGN / MIGRATE

Enterprise Database Investigation

Four technologies.
One question: what fits the business?

An illustrated guide to architecture decisions

Oracle / SQL Server
PostgreSQL / MySQL



Business scenarios and 3D hardware illustrations are conceptual. Product capabilities vary by edition and deployment.

Oracle

An established enterprise application can make continuity more valuable than a new platform.

Proprietary database



Where it fits

01

Banking, telecom, government and ERP.
Typical setting: certified applications, experienced DBAs and tested runbooks.



Availability design

02

RAC: multiple instances on a shared database.
Data Guard: standby continuity.
A single-site cluster cannot cover site loss.



Tools + connections

03

SQL Developer · Enterprise Manager · RMAN.
PL/SQL, JDBC and Oracle applications.
Deploy: on premises or cloud, including OCI.



Cost + dependency

04

Budget licenses, support, options and management packs.
Check actual entitlements.
PL/SQL dependencies can increase migration effort.



Oracle

RAC / Data Guard / RMAN



My view: choose Oracle when application fit and proven operations justify the full architecture cost.

SQL Server

Existing Microsoft skills, applications and reporting can make staying a rational choice.

Proprietary · edition matters



Where it fits

01

Finance, healthcare, manufacturing, education and public services.

Typical setting: .NET apps, reporting and a SQL Server operations team.



Strengths + availability

02

T-SQL, query optimization, indexing and analytics.

Availability groups support continuity; features vary by edition.

SQL Server also runs on Linux.



Tools + connections

03

SSMS · Integration Services · Analysis Services.

Microsoft identity and .NET integration.

Cloud route: Azure SQL Managed Instance; check compatibility.



Cost + dependency

04

Paid production editions plus restricted free editions.

Developer is for development/testing; Express has limits.

Budget licenses and Microsoft-specific dependencies.



SQL Server

.NET / T-SQL / Availability groups



My view: compare license cost with the real cost of rewriting integrations, retraining people and changing support.

PostgreSQL

A flexible relational core with licensing freedom and substantial engineering capability.

Permissive open-source license



Where it fits

01

SaaS, education, financial apps, government and spatial systems.

Typical setting: API-based products on Linux or managed cloud infrastructure.



Strengths + availability

02

Transactions, JSONB, rich indexes and extensions such as PostGIS.
Replication supplies building blocks; automatic failover needs an operations layer or managed service.



Tools + connections

03

pgAdmin · psql · pg_dump · pg_basebackup.
Java, Python and Node.js drivers.
Managed providers may restrict versions and extensions.



Cost + operating work

04

No core database license fee.
Budget hosting, specialist support and engineering time.
Maintain vacuum, statistics, connections, backups and upgrades.



PostgreSQL

SQL / JSONB / PostGIS



My view: a strong candidate for a new product when flexible queries and future options fit the team's skills.

MySQL

A familiar web application platform that can support serious transactional work.

GPL Community + commercial editions



Where it fits

01

E-commerce, publishing, web services and online platforms.

Typical setting: PHP, Java or Node.js storing users, products and orders.



Strengths + availability

02

InnoDB supports transactions and relational integrity.

InnoDB Cluster combines Group Replication, Shell and Router; single-primary is a common mode.



Tools + connections

03

MySQL Workbench · MySQL Shell · mysqldump.

Broad connector and application ecosystem.

Cloud: managed services, including RDS and HeatWave.



Cost + operating work

04

Community uses GPL; commercial editions add support and features.

Distribution terms need review.

A commercial application does not automatically require a paid license.



MySQL

InnoDB / Replication / InnoDB Cluster



My view: choose MySQL when workload fit and team experience are strong; test write contention and edition limits.

Different contexts. Different answers.

A recommendation connects a business constraint to a benefit, a trade-off and a test.

**BANK****Certified core system****Oracle may fit****BENEFIT**Application continuity
+ practiced recovery**Verify:** Prove failover and restore; confirm application support.**TRADE-OFF**License + option cost
and migration dependency**STARTUP****Cash + product changes****PostgreSQL may fit****BENEFIT**No core license fee
+ flexible queries**Verify:** Test the hardest queries and the total operating cost.**TRADE-OFF**Operations still cost;
team skills may favor MySQL.**MICROSOFT ESTATE****.NET + reporting****SQL Server may fit****BENEFIT**Reuse integrations,
people and support**Verify:** Compare staying with the full rewrite and retraining effort.**TRADE-OFF**Licenses and platform
dependencies remain.**Transfer rule:** debit account A and credit account B must succeed together or both roll back.

Oracle and MySQL with InnoDB both support transactions; recovery, compatibility and operating practice distinguish the choice.

Start with the workload, then the industry.

Representative fits, not exclusive sectors or market-share rankings.

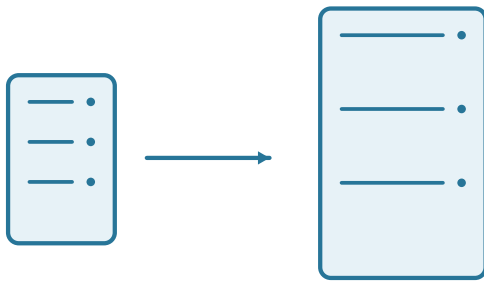


Large transactional systems: any of the four can fit. Test consistency, concurrency, storage latency and recovery.

Three growth patterns, three different jobs.

Performance is current behavior. Scalability is how that behavior changes with more work.

01 / SCALE UP

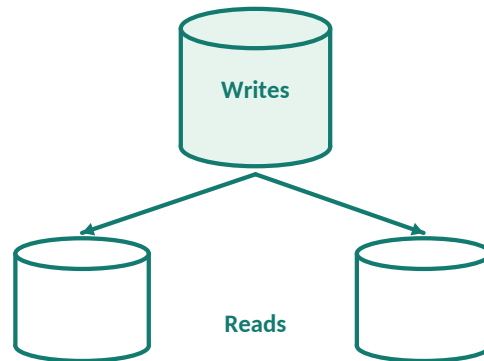


More power in one server

Add CPU, RAM or faster storage.

Trade-off: hardware ceilings, cost and a remaining failure domain.

02 / READ REPLICAS

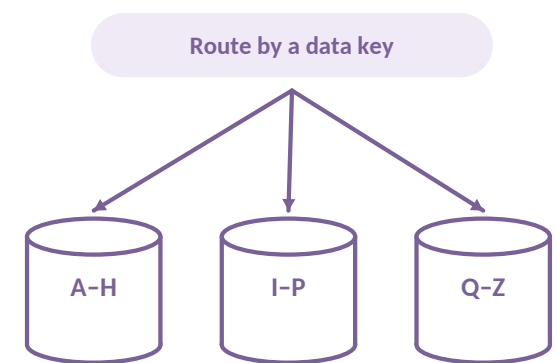


Spread the read workload

Replicas can also support continuity.

Trade-off: lag and routing. More replicas do not remove all write limits.

03 / SHARD WRITES



Distribute data + write work

Each shard owns a subset of the data.

Trade-off: cross-shard joins, transactions and rebalancing are harder.



Test the real workload

Representative data + concurrency
Comparable durability and hardware



Throughput

Work completed



p95 latency

95% finish by this time



Lock waits

Contention



Resources

CPU · I/O · memory

A cloud database still needs an owner.

Choose both an engine and an operating arrangement; managed features vary by service.

Your team owns

-  Schema + business rules
-  Access policies + data handling
-  Queries + application behavior



A provider may handle

-  Infrastructure + routine maintenance
-  Automated backups + retention
-  Failover + service monitoring

Check regions, data residency, app-to-database latency and service limits.
Agree recovery targets, maintenance windows and escalation responsibilities.

Technology

License / subscription
Compute / storage / I/O

People

Operations / training
Specialist support

Continuity

Support response terms
Outage exposure

Future change

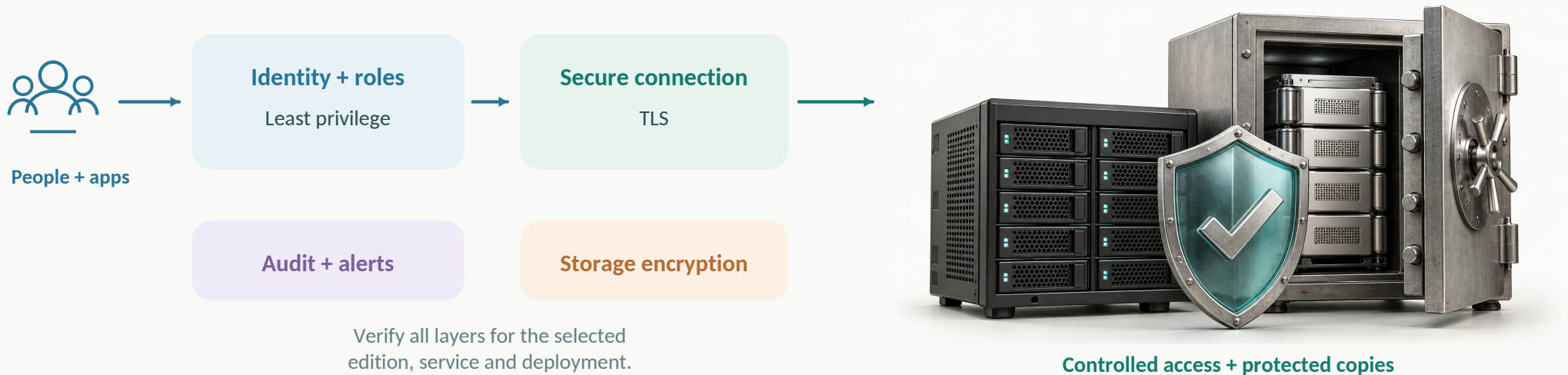
Migration / network transfer
Dependencies / exit cost



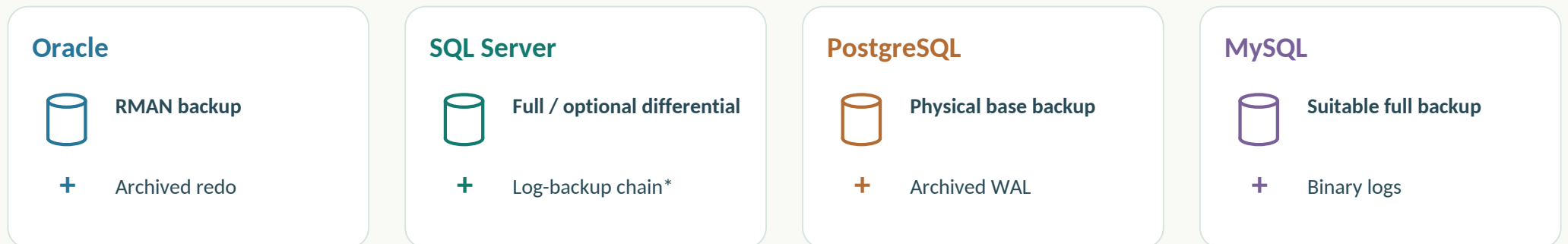
Why pay? Certified compatibility, features and escalation can justify cost. Open-source systems can also buy support.

Protect the data and protect a recoverable copy.

Access control, audit and backup design must work together.



BACKUP + RETAINED CHANGE LOGS = A PATH TO AN EARLIER STATE



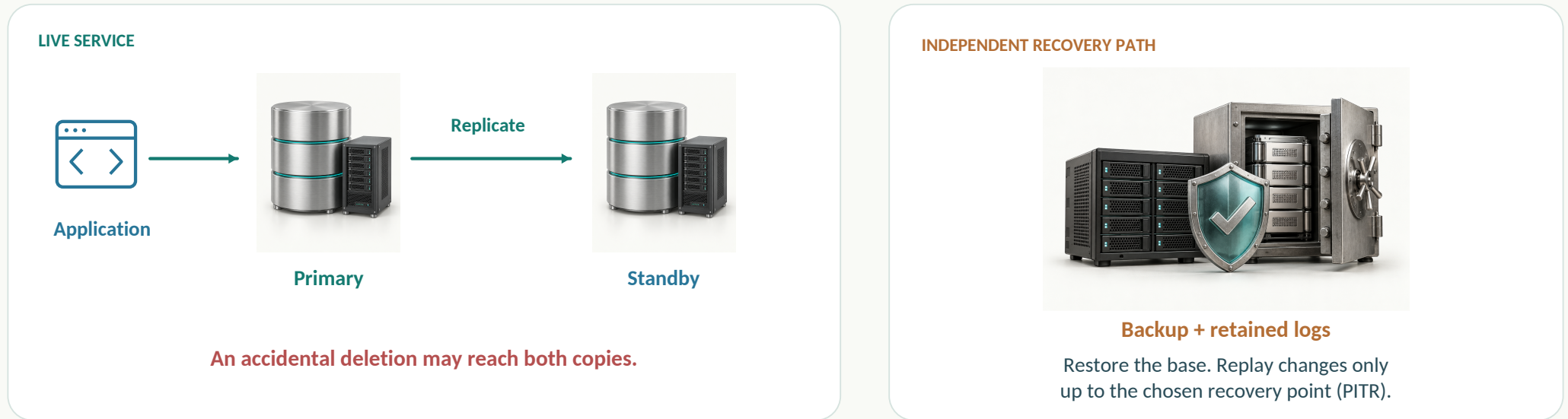
* SQL Server PITR needs a suitable recovery model and intact log chain. A PostgreSQL pg_dump is not a base for WAL replay.



Protect backups: encrypt copies, separate credentials, retain outside the primary failure domain and rehearse a verified restore.

A replica keeps up. A backup lets you go back.

Availability and point-in-time recovery solve different failure scenarios.



ILLUSTRATIVE OBJECTIVES — TIMELINE NOT TO SCALE



 **Verify the promise:** synchronous replication depends on acknowledgments, promotion rules and failure assumptions. Test end to end.

Shared SQL does not mean identical behavior.

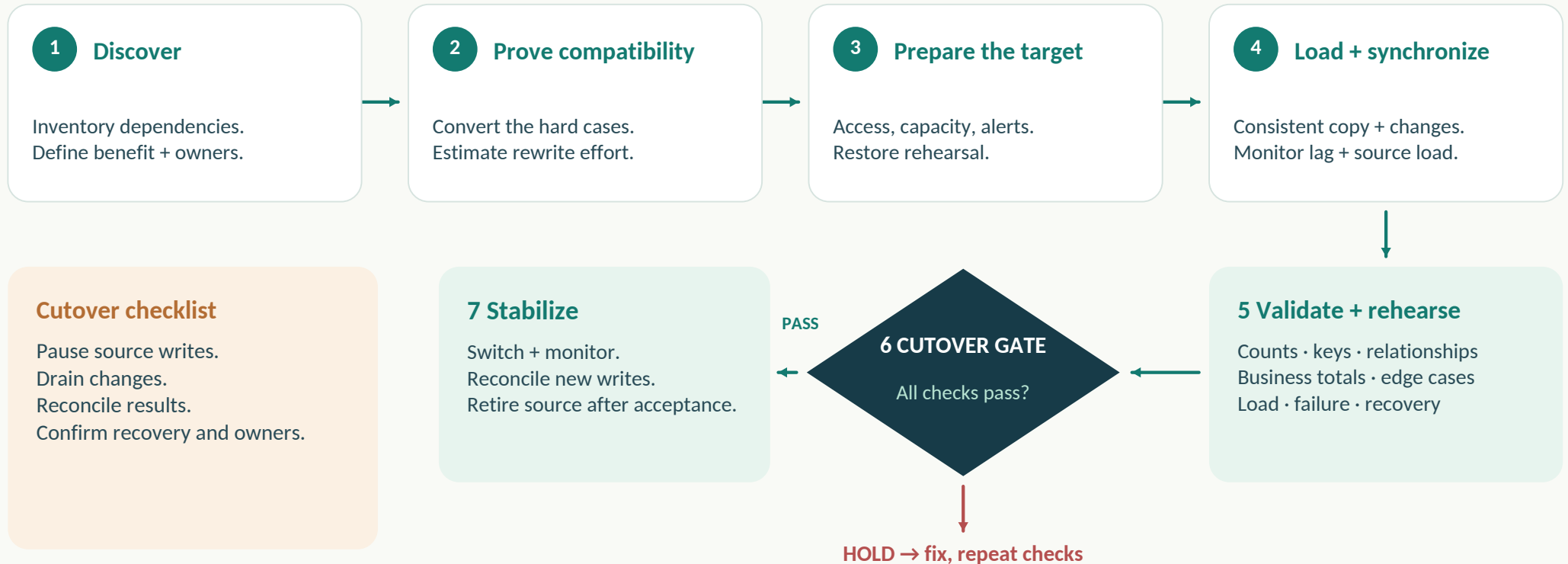
Tables, joins, procedures, triggers and transactions hide different implementations.



Why move? Cost pressure • cloud strategy • consolidation • new capabilities • support lifecycle.

A safe cutover is earned before the switch.

Set business owners, acceptance criteria and rollback conditions before moving production traffic.



Rollback has two different states



Keeping the old database is only part of the plan.

Before target writes

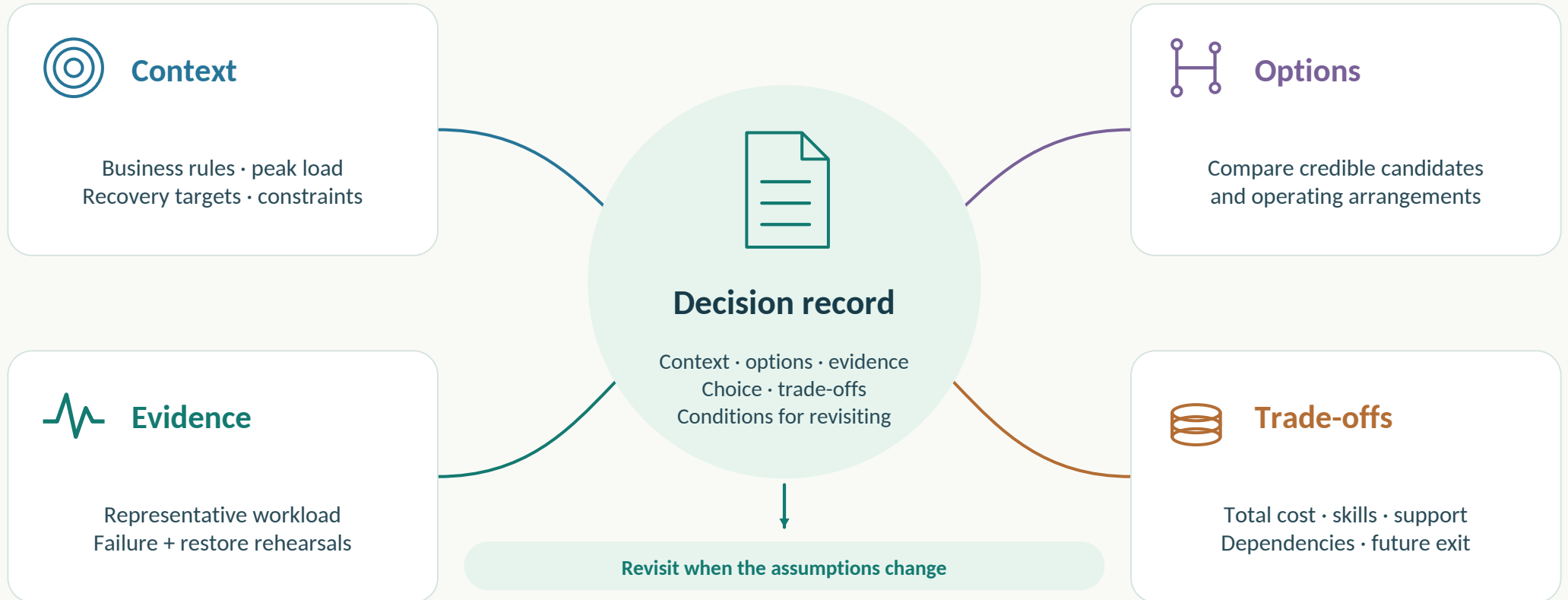
Return to source using the rehearsed switching plan.

After target writes

Reconcile or reverse changes, or apply a planned forward fix.

Choose the system you can keep reliable.

The best DBMS fits the workload, the people, the budget and the existing applications.



My engineering conclusion

Reliability is an ongoing responsibility.

I would test two credible candidates, prove recovery and compare the full operating cost. Then I would document the choice and its dependencies, so future engineers can change direction with evidence.